

HW3

2025-04-07

Problem 1 #Part a

```
library(MASS)
library(ggplot2)
data(Boston)
head(Boston)
```

```
##      crim zn indus chas   nox    rm  age    dis rad tax ptratio  black lstat
## 1 0.00632 18  2.31    0 0.538 6.575 65.2 4.0900   1 296    15.3 396.90  4.98
## 2 0.02731  0  7.07    0 0.469 6.421 78.9 4.9671   2 242    17.8 396.90  9.14
## 3 0.02729  0  7.07    0 0.469 7.185 61.1 4.9671   2 242    17.8 392.83  4.03
## 4 0.03237  0  2.18    0 0.458 6.998 45.8 6.0622   3 222    18.7 394.63  2.94
## 5 0.06905  0  2.18    0 0.458 7.147 54.2 6.0622   3 222    18.7 396.90  5.33
## 6 0.02985  0  2.18    0 0.458 6.430 58.7 6.0622   3 222    18.7 394.12  5.21
##   medv
## 1 24.0
## 2 21.6
## 3 34.7
## 4 33.4
## 5 36.2
## 6 28.7
```

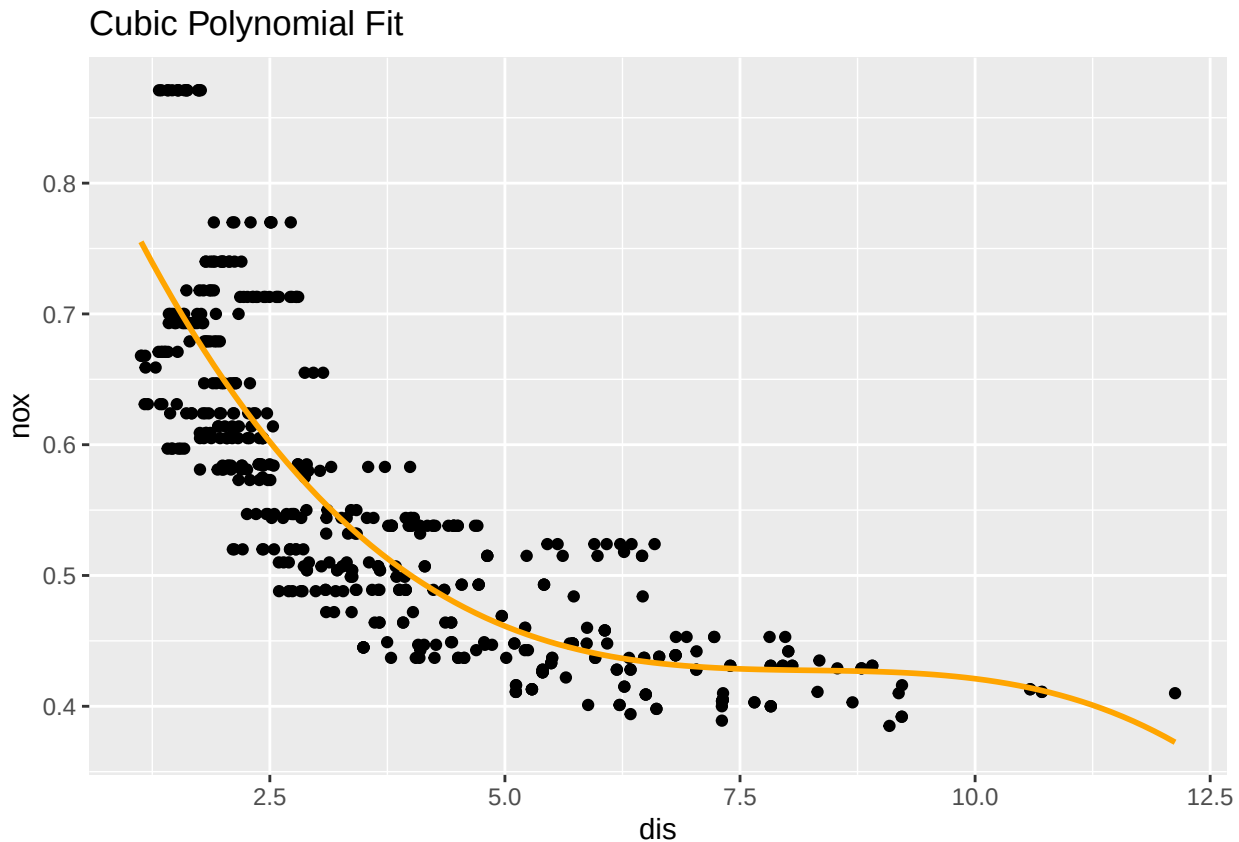
```
pomodel <- lm(nox ~ poly(dis, 3), data = Boston)
disSeq <- seq(min(Boston$dis), max(Boston$dis), length.out = 100)
predDF <- data.frame(dis = disSeq, pred = predict(pomodel, newdata = data.frame(dis = disSeq)))
summary(pomodel)
```

```
##
## Call:
## lm(formula = nox ~ poly(dis, 3), data = Boston)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -0.121130 -0.040619 -0.009738  0.023385  0.194904
##
## Coefficients:
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept)   0.554695   0.002759  201.021 < 2e-16 ***
## poly(dis, 3)1 -2.003096   0.062071  -32.271 < 2e-16 ***
## poly(dis, 3)2  0.856330   0.062071   13.796 < 2e-16 ***
## poly(dis, 3)3 -0.318049   0.062071   -5.124 4.27e-07 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

```
##
## Residual standard error: 0.06207 on 502 degrees of freedom
## Multiple R-squared:  0.7148, Adjusted R-squared:  0.7131
## F-statistic: 419.3 on 3 and 502 DF,  p-value: < 2.2e-16
```

```
ggplot(Boston, aes(x = dis, y = nox)) +
  geom_point() +
  geom_line(data = predDF, aes(x = dis, y = pred), color = "orange", size = 1) +
  ggtitle("Cubic Polynomial Fit") +
  xlab("dis") +
  ylab("nox")
```

```
## Warning: Using 'size' aesthetic for lines was deprecated in ggplot2 3.4.0.
## i Please use 'linewidth' instead.
## This warning is displayed once every 8 hours.
## Call 'lifecycle::last_lifecycle_warnings()' to see where this warning was
## generated.
```



The model explains about 71.5% of the variance in nox (Multiple R-squared = 0.7148), and the overall F-test p ($< 2.2e-16$) shows that the model is statistically significant. From the plot, we see trend is strongly negative at lower values of dis and higher nox, then it gradually flattens out and tapers off toward the end. We can see that a third degree polynomial can capture patterns of data except at extremes. #Part b

```
allPoly <- data.frame()
rssVals <- numeric(10)
disSeq <- seq(min(Boston$dis), max(Boston$dis), length.out = 100)
```

```

for (deg in 1:10) {
  mod <- lm(nox ~ poly(dis, deg), data = Boston)
  rssVals[deg] <- sum(residuals(mod)^2)
  tempDF <- data.frame(dis = disSeq,
                        pred = predict(mod, newdata = data.frame(dis = disSeq)),
                        degree = as.factor(deg))
  allPoly <- rbind(allPoly, tempDF)
}
print(rssVals)

```

```

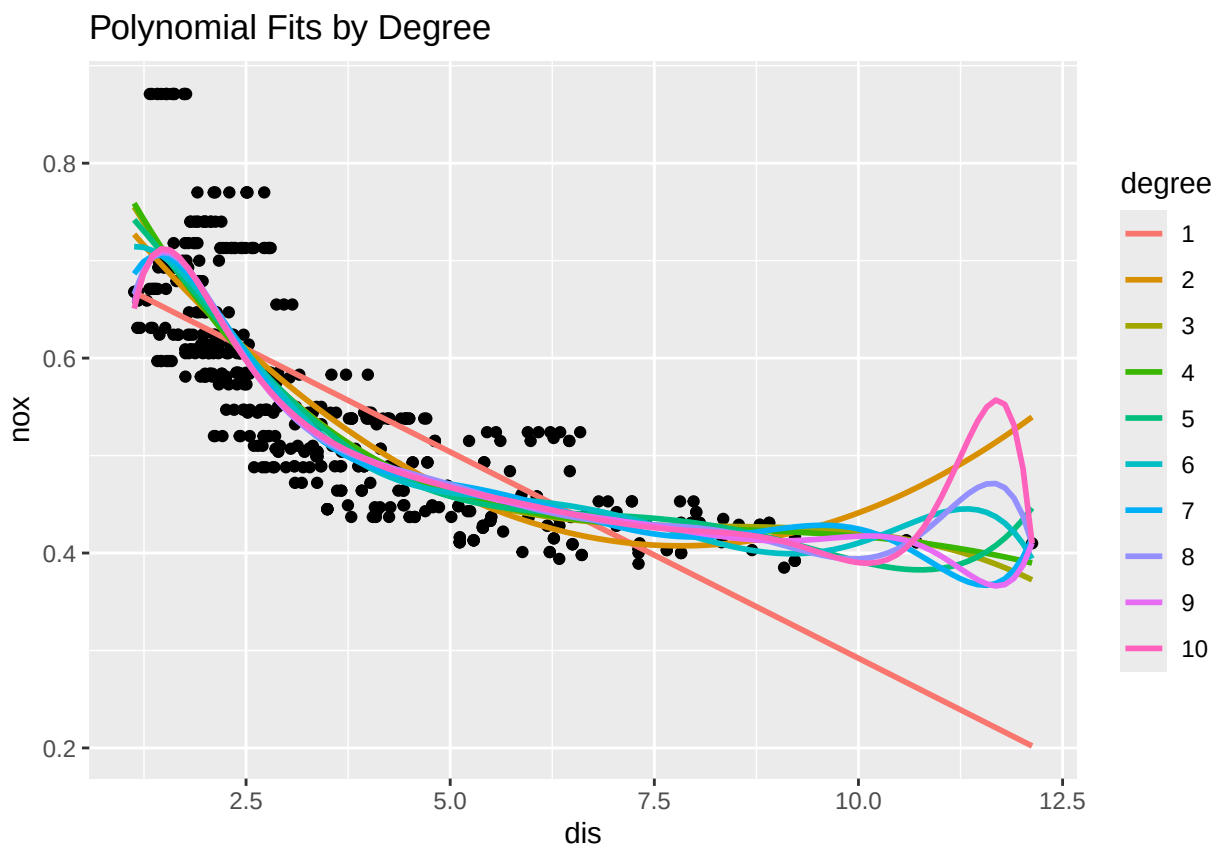
## [1] 2.768563 2.035262 1.934107 1.932981 1.915290 1.878257 1.849484 1.835630
## [9] 1.833331 1.832171

```

```

ggplot(Boston, aes(x = dis, y = nox)) +
  geom_point() +
  geom_line(data = allPoly, aes(x = dis, y = pred, color = degree), size = 1) +
  ggtitle("Polynomial Fits by Degree") +
  xlab("dis") +
  ylab("nox")

```



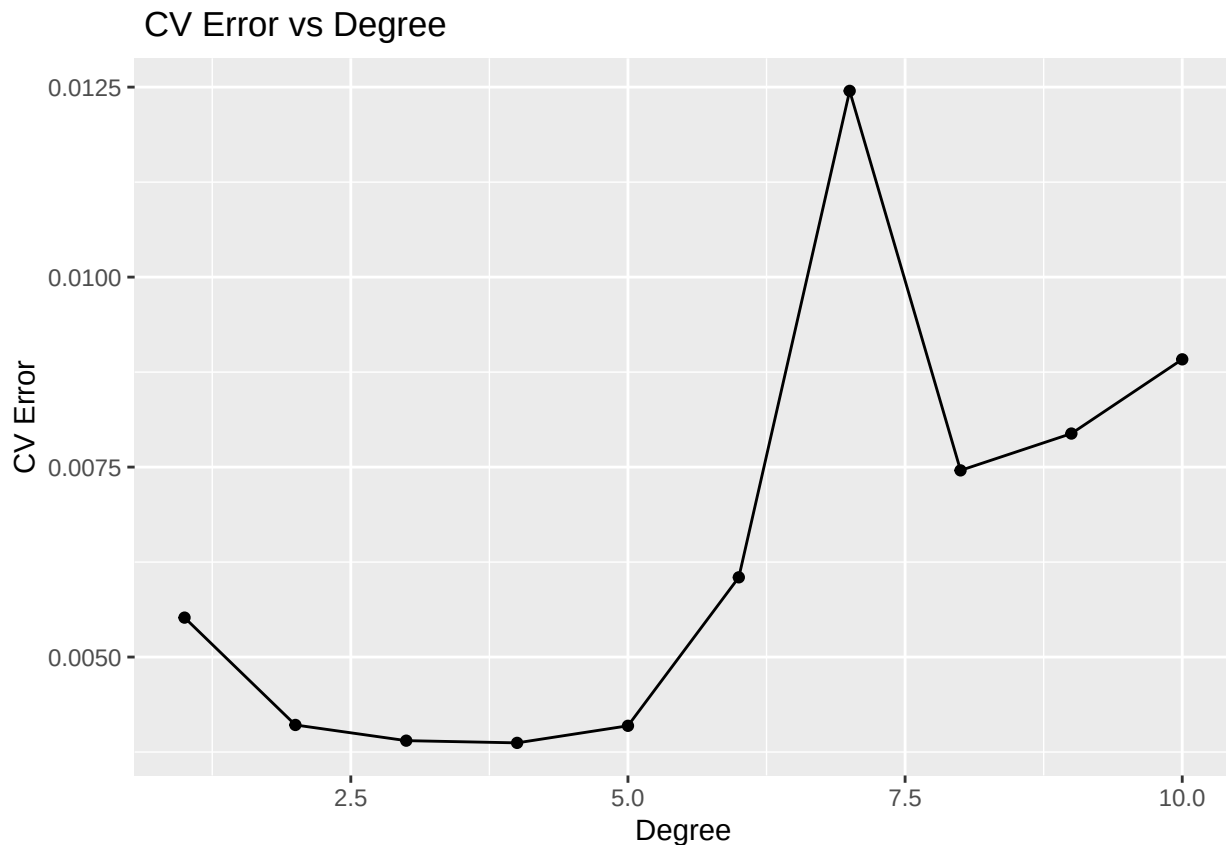
As the polynomial degree increases, the RSS decreases, indicating closer fit to the data. The difference between RSS of degrees 3 and 4 is very small and following degrees have even smaller RSS difference. So, for higher degrees the improvement is not much. higher degree is leading to overfitting, so we need to balance complexity without overfitting.

#Part c

```
library(boot)
cvErrors <- numeric(10)
set.seed(42)
for (deg in 1:10) {
  modCV <- glm(nox ~ poly(dis, deg), data = Boston)
  cvRes <- cv.glm(Boston, modCV, K = 10)
  cvErrors[deg] <- cvRes$delta[1]
}
cvDF <- data.frame(Degree = 1:10, CVError = cvErrors)
print(cvErrors)
```

```
## [1] 0.005518839 0.004106439 0.003899815 0.003870615 0.004095242 0.006049225
## [7] 0.012450722 0.007456502 0.007941055 0.008917719
```

```
ggplot(cvDF, aes(x = Degree, y = CVError)) +
  geom_point() +
  geom_line() +
  ggtitle(" CV Error vs Degree") +
  xlab("Degree") +
  ylab("CV Error")
```

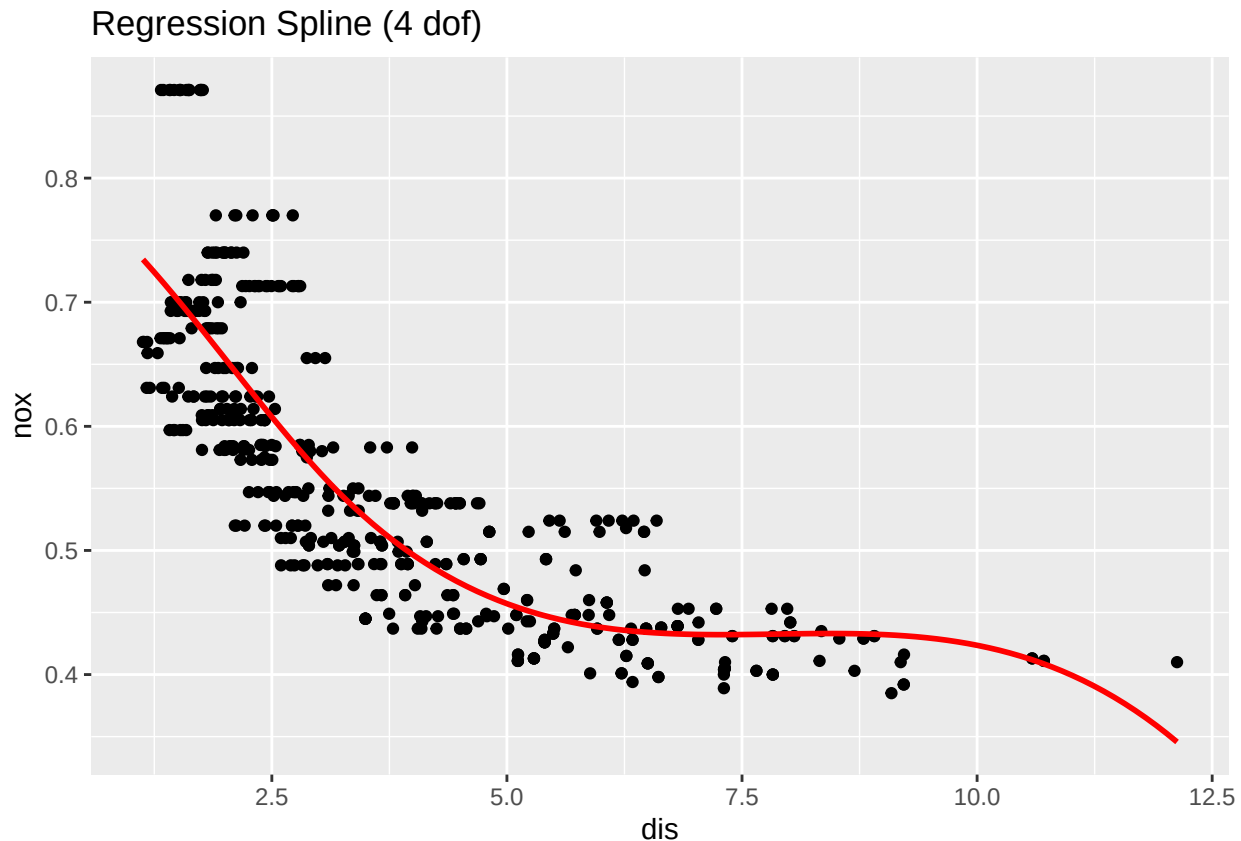


The CV error starts at 0.0055 (approx) for the first degree, then it keeps decreasing as we increase degree and reaches it's lowest point at fourth degree. This shows that fourth degree polynomial model provides the best balance with model complexity and performance (cv error). After degree 4, the CV error increases as we increase the degrees in the model, reaching peak at degree 7, indicating that higher degree polynomials are likely overfitting the data. After degree 7, the cv error decreases but still is significantly high. #Part d

```
library(splines)
sModel <- lm(nox ~ bs(dis, df = 4), data = Boston)
splineDF <- data.frame(dis = disSeq, pred = predict(sModel, newdata = data.frame(dis = disSeq)))
summary(sModel)
```

```
##
## Call:
## lm(formula = nox ~ bs(dis, df = 4), data = Boston)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -0.124622 -0.039259 -0.008514  0.020850  0.193891
##
## Coefficients:
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept)    0.73447    0.01460  50.306 < 2e-16 ***
## bs(dis, df = 4)1 -0.05810    0.02186  -2.658  0.00812 **
## bs(dis, df = 4)2 -0.46356    0.02366 -19.596 < 2e-16 ***
## bs(dis, df = 4)3 -0.19979    0.04311  -4.634  4.58e-06 ***
## bs(dis, df = 4)4 -0.38881    0.04551  -8.544 < 2e-16 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 0.06195 on 501 degrees of freedom
## Multiple R-squared:  0.7164, Adjusted R-squared:  0.7142
## F-statistic: 316.5 on 4 and 501 DF,  p-value: < 2.2e-16
```

```
ggplot(Boston, aes(x = dis, y = nox)) +
  geom_point() +
  geom_line(data = splineDF, aes(x = dis, y = pred), color = "red", size = 1) +
  ggtitle("Regression Spline (4 dof)") +
  xlab("dis") +
  ylab("nox")
```



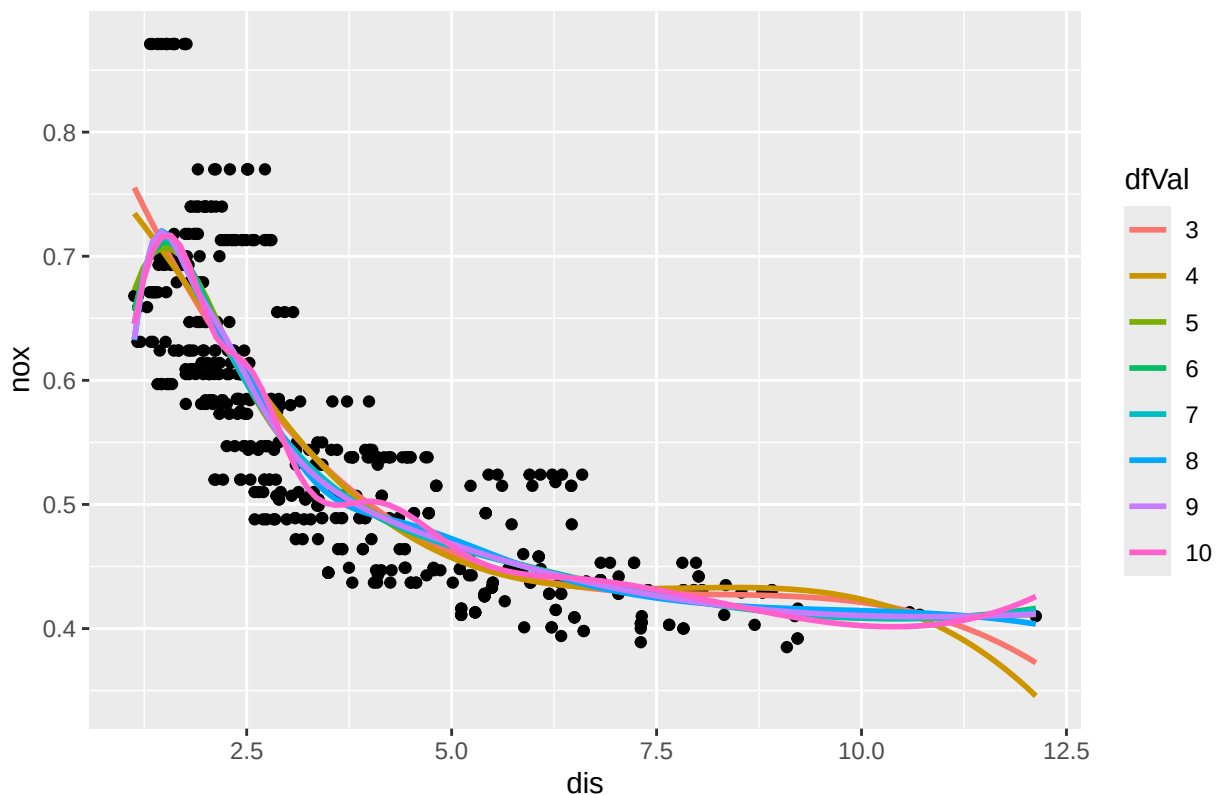
The residual standard error is 0.06195, and the F-statistic is 316.5 ($p < 2e-16$), this shows that the model is statistically significant. We used 4 degrees of freedom, the `bs()` function by default chooses knot positions that divide the data range into sections with roughly equal numbers of observations. The plot confirms that the spline captures the relationship between `dis` and `nox` well. #Part e

```
dfList <- 3:10
splineAll <- data.frame()
splineRSS <- numeric(length(dfList))
for (i in seq_along(dfList)) {
  modSpline <- lm(nox ~ bs(dis, df = dfList[i]), data = Boston)
  splineRSS[i] <- sum(residuals(modSpline)^2)
  tempDF <- data.frame(dis = disSeq,
                       pred = predict(modSpline, newdata = data.frame(dis = disSeq)),
                       dfVal = as.factor(dfList[i]))
  splineAll <- rbind(splineAll, tempDF)
}
print(splineRSS)
```

```
## [1] 1.934107 1.922775 1.840173 1.833966 1.829884 1.816995 1.825653 1.792535
```

```
ggplot(Boston, aes(x = dis, y = nox)) +
  geom_point() +
  geom_line(data = splineAll, aes(x = dis, y = pred, color = dfVal), size = 1) +
  ggtitle("Spline Fits with range of dof") +
  xlab("dis") +
  ylab("nox")
```

Spline Fits with range of dof

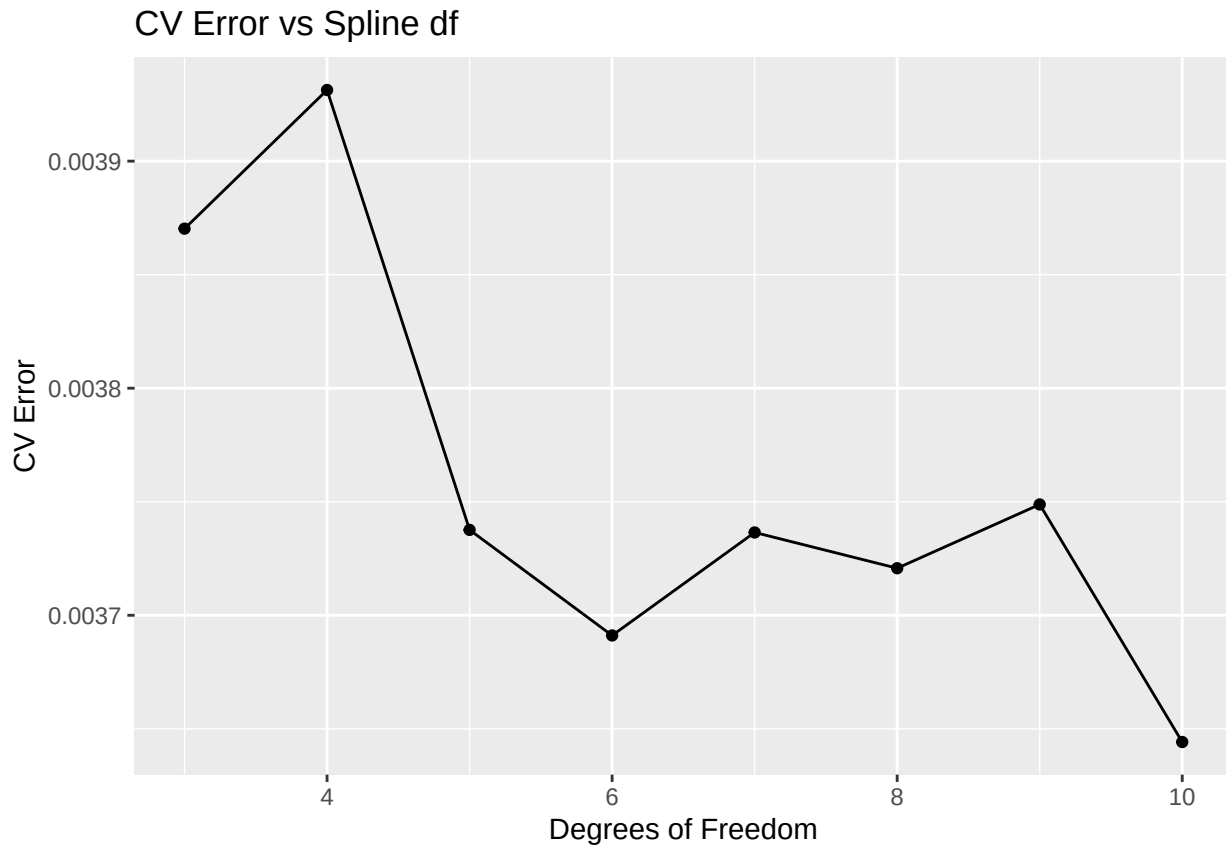


As the spline's degrees of freedom increase from 3 to 10, the RSS declines. This shows that adding more degrees of freedom gives the spline enough flexibility to capture variations in the data, thus reducing the overall error. However, at dof 9 and 10, difference in RSS is small and fluctuates so further increasing will capture noise rather than the data. #Part f

```
cvSplineErrors <- numeric(length(dfList))
set.seed(42)
for (i in seq_along(dfList)) {
  modSp <- glm(nox ~ bs(dis, df = dfList[i]), data = Boston)
  cvResSp <- suppressWarnings(cv.glm(Boston, modSp, K = 10))
  cvSplineErrors[i] <- cvResSp$delta[1]
}
cvSplineDF <- data.frame(dfVal = dfList, CLError = cvSplineErrors)
print(cvSplineErrors)
```

```
## [1] 0.003870257 0.003931326 0.003737600 0.003691096 0.003736470 0.003720710
## [7] 0.003748807 0.003644179
```

```
ggplot(cvSplineDF, aes(x = dfVal, y = CLError)) +
  geom_point() +
  geom_line() +
  ggtitle("CV Error vs Spline df") +
  xlab("Degrees of Freedom") +
  ylab("CV Error")
```



The best CV error is with 10 degrees of freedom (CV error = 0.00364), but the cv error differences between 6 and 10 are small. This means adding extra dof has slight improvement in the fit, it might cause overfitting.

Problem 2 #Part a

```
library(ISLR)
library(leaps)
collegeData <- College
head(collegeData)
```

```
##               Private Apps Accept Enroll Top10perc Top25perc
## Abilene Christian University    Yes 1660  1232    721         23         52
## Adelphi University             Yes 2186  1924    512         16         29
## Adrian College                 Yes 1428  1097    336         22         50
## Agnes Scott College            Yes  417   349    137         60         89
## Alaska Pacific University       Yes  193   146     55         16         44
## Albertson College              Yes  587   479    158         38         62
##
##               F.Undergrad P.Undergrad Outstate Room.Board Books
## Abilene Christian University    2885         537    7440     3300    450
## Adelphi University              2683        1227   12280     6450    750
## Adrian College                  1036          99   11250     3750    400
## Agnes Scott College              510          63   12960     5450    450
## Alaska Pacific University        249         869    7560     4120    800
## Albertson College                678          41   13500     3335    500
##
##               Personal PhD Terminal S.F.Ratio perc.alumni Expend
## Abilene Christian University    2200    70         78     18.1         12    7041
## Adelphi University              1500    29         30     12.2         16   10527
## Adrian College                  1165    53         66     12.9         30    8735
```

## Agnes Scott College	875	92	97	7.7	37	19016
## Alaska Pacific University	1500	76	72	11.9	2	10922
## Albertson College	675	67	73	9.4	11	9727
##	Grad.Rate					
## Abilene Christian University	60					
## Adelphi University	56					
## Adrian College	54					
## Agnes Scott College	59					
## Alaska Pacific University	15					
## Albertson College	55					

```

set.seed(1)
nTotal <- nrow(collegeData)
trainIdx <- sample(1:nTotal, size = round(0.7 * nTotal))
train <- collegeData[trainIdx, ]
test <- collegeData[-trainIdx, ]

fwdModel <- regsubsets(Outstate ~ ., data = train, nvmax = ncol(train) - 1, method = "forward")
fwdSum <- summary(fwdModel)
print(fwdSum)

```

```

## Subset selection object
## Call: regsubsets.formula(Outstate ~ ., data = train, nvmax = ncol(train) -
##      1, method = "forward")
## 17 Variables (and intercept)
##              Forced in Forced out
## PrivateYes      FALSE      FALSE
## Apps            FALSE      FALSE
## Accept          FALSE      FALSE
## Enroll          FALSE      FALSE
## Top10perc       FALSE      FALSE
## Top25perc       FALSE      FALSE
## F.Undergrad     FALSE      FALSE
## P.Undergrad     FALSE      FALSE
## Room.Board     FALSE      FALSE
## Books           FALSE      FALSE
## Personal        FALSE      FALSE
## PhD            FALSE      FALSE
## Terminal        FALSE      FALSE
## S.F.Ratio       FALSE      FALSE
## perc.alumni     FALSE      FALSE
## Expend          FALSE      FALSE
## Grad.Rate       FALSE      FALSE
## 1 subsets of each size up to 17
## Selection Algorithm: forward
##              PrivateYes Apps Accept Enroll Top10perc Top25perc F.Undergrad
## 1  ( 1 )  " "          " " " "    " "    " "          " "
## 2  ( 1 )  "*"          " " " "    " "    " "          " "
## 3  ( 1 )  "*"          " " " "    " "    " "          " "
## 4  ( 1 )  "*"          " " " "    " "    " "          " "
## 5  ( 1 )  "*"          " " " "    " "    " "          " "
## 6  ( 1 )  "*"          " " " "    " "    " "          " "
## 7  ( 1 )  "*"          " " " "    " "    " "          " "
## 8  ( 1 )  "*"          " " " "    " "    " "          " "

```

```

## 9 ( 1 ) "*" " " " " " " "*" " " " "
## 10 ( 1 ) "*" " " " " " " "*" " " " "
## 11 ( 1 ) "*" " " "*" " " "*" " " " "
## 12 ( 1 ) "*" "*" "*" " " "*" " " " "
## 13 ( 1 ) "*" "*" "*" " " "*" " " "*"
## 14 ( 1 ) "*" "*" "*" " " "*" "*" "*"
## 15 ( 1 ) "*" "*" "*" " " "*" "*" "*"
## 16 ( 1 ) "*" "*" "*" " " "*" "*" "*"
## 17 ( 1 ) "*" "*" "*" "*" "*" "*" "*"
##
## P.Undergrad Room.Board Books Personal PhD Terminal S.F.Ratio
## 1 ( 1 ) " " " " " " " " " "
## 2 ( 1 ) " " " " " " " " " "
## 3 ( 1 ) " " "*" " " " " " " " "
## 4 ( 1 ) " " "*" " " " " " " " "
## 5 ( 1 ) " " "*" " " " " "*" " " " "
## 6 ( 1 ) " " "*" " " " " "*" " " " "
## 7 ( 1 ) " " "*" " " "*" "*" " " " "
## 8 ( 1 ) " " "*" " " "*" "*" "*" " " "
## 9 ( 1 ) " " "*" " " "*" "*" "*" " " "
## 10 ( 1 ) " " "*" " " "*" "*" "*" "*"
## 11 ( 1 ) " " "*" " " "*" "*" "*" "*"
## 12 ( 1 ) " " "*" " " "*" "*" "*" "*"
## 13 ( 1 ) " " "*" " " "*" "*" "*" "*"
## 14 ( 1 ) " " "*" " " "*" "*" "*" "*"
## 15 ( 1 ) "*" "*" " " "*" "*" "*" "*"
## 16 ( 1 ) "*" "*" "*" "*" "*" "*" "*"
## 17 ( 1 ) "*" "*" "*" "*" "*" "*" "*"
##
## perc.alumni Expend Grad.Rate
## 1 ( 1 ) " " "*" " "
## 2 ( 1 ) " " "*" " "
## 3 ( 1 ) " " "*" " "
## 4 ( 1 ) "*" "*" " "
## 5 ( 1 ) "*" "*" " "
## 6 ( 1 ) "*" "*" "*"
## 7 ( 1 ) "*" "*" "*"
## 8 ( 1 ) "*" "*" "*"
## 9 ( 1 ) "*" "*" "*"
## 10 ( 1 ) "*" "*" "*"
## 11 ( 1 ) "*" "*" "*"
## 12 ( 1 ) "*" "*" "*"
## 13 ( 1 ) "*" "*" "*"
## 14 ( 1 ) "*" "*" "*"
## 15 ( 1 ) "*" "*" "*"
## 16 ( 1 ) "*" "*" "*"
## 17 ( 1 ) "*" "*" "*"

```

```

bestIdx <- which.min(fwdSum$bic)
cat("Best model has", bestIdx, "predictors.\n")

```

```

## Best model has 13 predictors.

```

```

print(fwdSum$which[bestIdx, ])

```

```
## (Intercept) PrivateYes Apps Accept Enroll Top10perc
## TRUE TRUE TRUE TRUE FALSE TRUE
## Top25perc F.Undergrad P.Undergrad Room.Board Books Personal
## FALSE TRUE FALSE TRUE FALSE TRUE
## PhD Terminal S.F.Ratio perc.alumni Expend Grad.Rate
## TRUE TRUE TRUE TRUE TRUE TRUE
```

This shows the model with lowest BIC has 13 predictors. This model has variables like PrivateYes, Apps, Accept, Top10perc, F.Undergrad, Room.Board, Personal, PhD, Terminal, S.F.Ratio, perc.alumni, Expend, and Grad.Rate. #Part b

```
#we chose only the key predictors top10perc, accept and enroll, as they provide most useful information
library(mgcv)
```

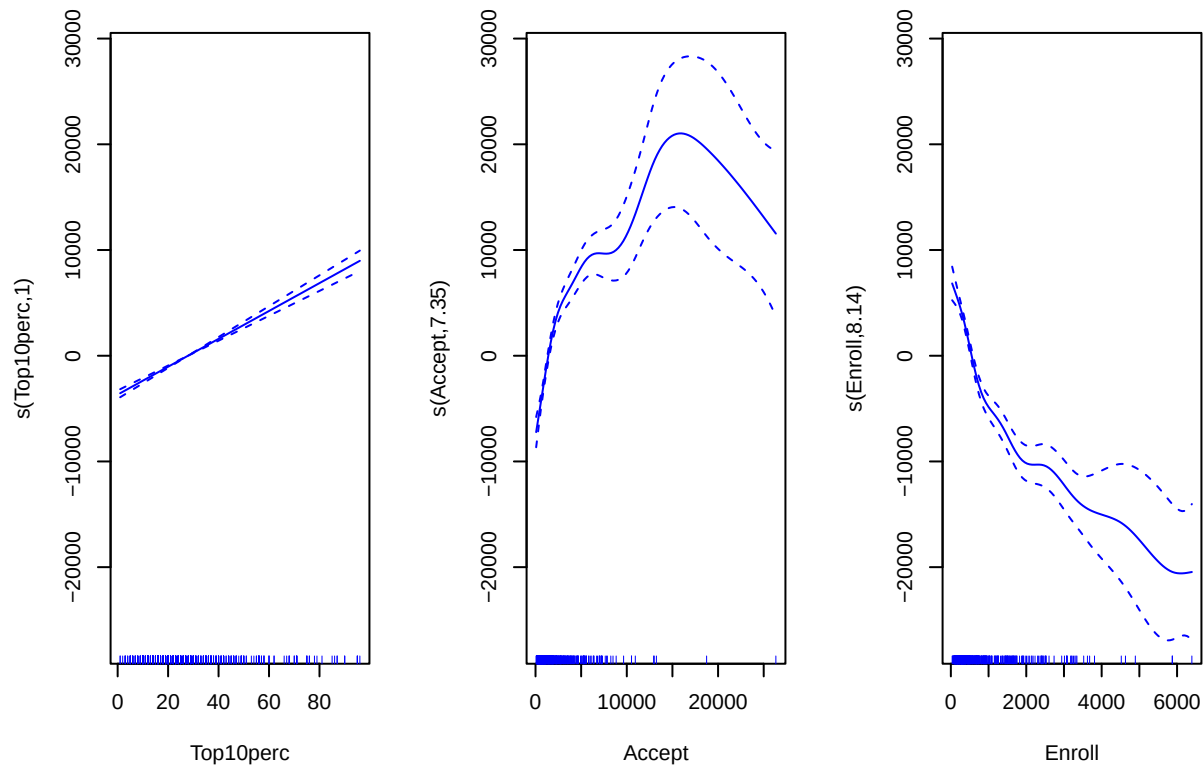
```
## Loading required package: nlme
```

```
## This is mgcv 1.9-1. For overview type 'help("mgcv-package")'.
```

```
gmodel <- gam(Outstate ~ s(Top10perc) + s(Accept) + s(Enroll), data = train)
summary(gmodel)
```

```
##
## Family: gaussian
## Link function: identity
##
## Formula:
## Outstate ~ s(Top10perc) + s(Accept) + s(Enroll)
##
## Parametric coefficients:
##             Estimate Std. Error t value Pr(>|t|)
## (Intercept) 10632.4      119.1   89.25  <2e-16 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Approximate significance of smooth terms:
##             edf Ref.df      F p-value
## s(Top10perc) 1.000  1.000 346.31  <2e-16 ***
## s(Accept)     7.350  8.348  21.42  <2e-16 ***
## s(Enroll)     8.141  8.771  25.76  <2e-16 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## R-sq.(adj) =  0.547   Deviance explained = 56.1%
## GCV = 7.9765e+06   Scale est. = 7.72e+06   n = 544
```

```
par(mfrow = c(1, 3))
plot(gmodel, se = TRUE, col = "blue")
```



```
par(mfrow = c(1, 1))
```

The model has an adjusted R-squared of 0.547 and it explains about 56% of the deviance, that means the model captures a good portion of the tuition variation. $p < 2e-16$, this shows that each predictor is statistically significant. Top10perc behaves almost linearly, Accept and Enroll show complex, non-linear plots.

#Part c

```
gPred <- predict(gmodel, newdata = test)
mse <- mean((test$Outstate - gPred)^2)
cat("Test MSE:", mse, "\n")
```

```
## Test MSE: 6907053
```

#Part d

```
summary(gmodel)
```

```
##
## Family: gaussian
## Link function: identity
##
## Formula:
## Outstate ~ s(Top10perc) + s(Accept) + s(Enroll)
##
## Parametric coefficients:
##               Estimate Std. Error t value Pr(>|t|)
## (Intercept)  10632.4      119.1    89.25  <2e-16 ***
```

```
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Approximate significance of smooth terms:
##           edf Ref.df      F p-value
## s(Top10perc) 1.000  1.000 346.31 <2e-16 ***
## s(Accept)     7.350  8.348  21.42 <2e-16 ***
## s(Enroll)     8.141  8.771  25.76 <2e-16 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## R-sq.(adj) =  0.547   Deviance explained = 56.1%
## GCV = 7.9765e+06   Scale est. = 7.72e+06   n = 544
```

The smooth term for Top10perc has an effective degree of freedom (edf) of 1, showing linear relationship, while the smooth terms for Accept (edf = 7.35) and Enroll (edf = 8.14) reveal complex, non-linear relationships. All smooth terms are statistically significant as $p < 2e-16$. Problem 3 #Part a

```
library(ISLR)
library(leaps)
data(OJ)
head(OJ)
```

```
##   Purchase WeekofPurchase StoreID PriceCH PriceMM DiscCH DiscMM SpecialCH
## 1      CH                237      1    1.75    1.99    0.00    0.0         0
## 2      CH                239      1    1.75    1.99    0.00    0.3         0
## 3      CH                245      1    1.86    2.09    0.17    0.0         0
## 4      MM                227      1    1.69    1.69    0.00    0.0         0
## 5      CH                228      7    1.69    1.69    0.00    0.0         0
## 6      CH                230      7    1.69    1.99    0.00    0.0         0
##   SpecialMM LoyalCH SalePriceMM SalePriceCH PriceDiff Store7 PctDiscMM
## 1          0 0.500000         1.99         1.75      0.24     No  0.000000
## 2          1 0.600000         1.69         1.75     -0.06     No  0.150754
## 3          0 0.680000         2.09         1.69      0.40     No  0.000000
## 4          0 0.400000         1.69         1.69      0.00     No  0.000000
## 5          0 0.956535         1.69         1.69      0.00     Yes 0.000000
## 6          1 0.965228         1.99         1.69      0.30     Yes 0.000000
##   PctDiscCH ListPriceDiff STORE
## 1 0.000000         0.24      1
## 2 0.000000         0.24      1
## 3 0.091398         0.23      1
## 4 0.000000         0.00      1
## 5 0.000000         0.00      0
## 6 0.000000         0.30      0
```

```
set.seed(1)
nRows <- nrow(OJ)
trainIdx <- sample(1:nRows, 800)
ojTrain <- OJ[trainIdx, ]
ojTest  <- OJ[-trainIdx, ]
```

#Part b

```
library(tree)
ojTree <- tree(Purchase ~ ., data = ojTrain)
summary(ojTree)
```

```
##
## Classification tree:
## tree(formula = Purchase ~ ., data = ojTrain)
## Variables actually used in tree construction:
## [1] "LoyalCH"      "PriceDiff"    "SpecialCH"    "ListPriceDiff"
## [5] "PctDiscMM"
## Number of terminal nodes: 9
## Residual mean deviance: 0.7432 = 587.8 / 791
## Misclassification error rate: 0.1588 = 127 / 800
```

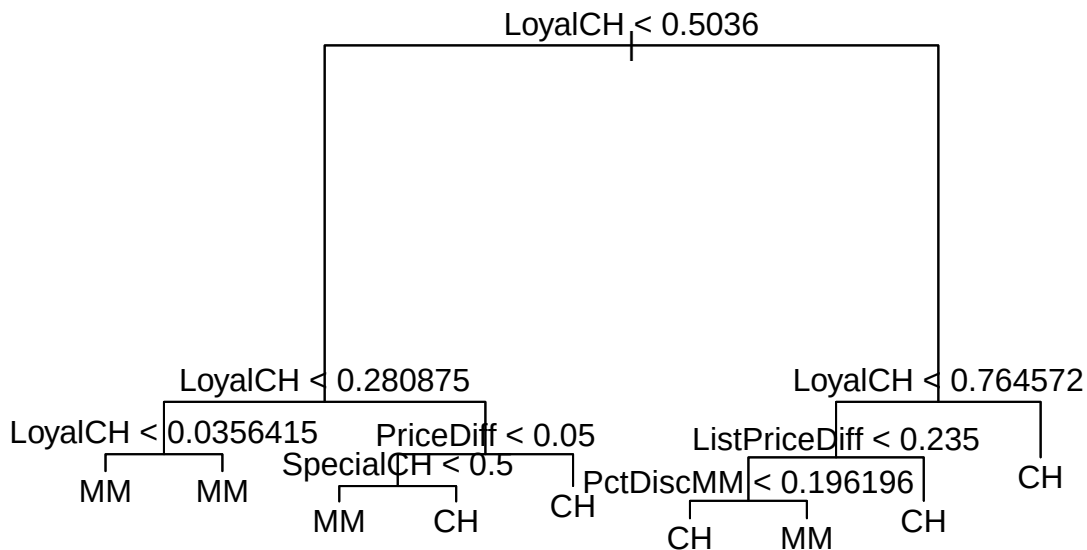
The tree has five variables LoyalCH, PriceDiff, SpecialCH, ListPriceDiff, and PctDiscMM (we didn't have Buy variable). It created 9 terminal nodes, the data is partitioned into 9 distinct groups. The residual mean deviance of 0.7432 (low) shows homogeneity. misclassification error (training error) is 15.88% meaning the tree correctly classifies about 84% of the training data. #Part c

```
ojTree
```

```
## node), split, n, deviance, yval, (yprob)
##      * denotes terminal node
##
## 1) root 800 1073.00 CH ( 0.60625 0.39375 )
##    2) LoyalCH < 0.5036 365 441.60 MM ( 0.29315 0.70685 )
##      4) LoyalCH < 0.280875 177 140.50 MM ( 0.13559 0.86441 )
##        8) LoyalCH < 0.0356415 59 10.14 MM ( 0.01695 0.98305 ) *
##        9) LoyalCH > 0.0356415 118 116.40 MM ( 0.19492 0.80508 ) *
##      5) LoyalCH > 0.280875 188 258.00 MM ( 0.44149 0.55851 )
##        10) PriceDiff < 0.05 79 84.79 MM ( 0.22785 0.77215 )
##          20) SpecialCH < 0.5 64 51.98 MM ( 0.14062 0.85938 ) *
##          21) SpecialCH > 0.5 15 20.19 CH ( 0.60000 0.40000 ) *
##        11) PriceDiff > 0.05 109 147.00 CH ( 0.59633 0.40367 ) *
##    3) LoyalCH > 0.5036 435 337.90 CH ( 0.86897 0.13103 )
##      6) LoyalCH < 0.764572 174 201.00 CH ( 0.73563 0.26437 )
##        12) ListPriceDiff < 0.235 72 99.81 MM ( 0.50000 0.50000 )
##          24) PctDiscMM < 0.196196 55 73.14 CH ( 0.61818 0.38182 ) *
##          25) PctDiscMM > 0.196196 17 12.32 MM ( 0.11765 0.88235 ) *
##        13) ListPriceDiff > 0.235 102 65.43 CH ( 0.90196 0.09804 ) *
##    7) LoyalCH > 0.764572 261 91.20 CH ( 0.95785 0.04215 ) *
```

The tree uses five variables—LoyalCH, PriceDiff, SpecialCH, ListPriceDiff, and PctDiscMM—to split the data into 9 distinct groups. It starts with 800 observations, predicting CH with about 61% probability. The first split on LoyalCH separates observations with values below 0.5036 (node 2 with 365 cases, leaning toward MM) from those above 0.5036 (node 3 with 435 cases, mostly CH). Within node 2, a further split on LoyalCH at 0.280875 creates smaller groups: one branch (node 8) with very low LoyalCH (below 0.0356415) almost always predicts MM, while another branch (node 9) still predicts MM strongly. Additionally, node 5 in this branch splits on PriceDiff and then on SpecialCH to decide between MM and CH. In node 3, further splits on LoyalCH, ListPriceDiff, and PctDiscMM narrows down the classifications, with node 7 predicting CH very well. Terminal nodes, marked with an asterisk, represent the final groups where no more splits occur. #Part d

```
plot(ojTree)
text(ojTree, pretty = 0)
```



This tree makes it clear that LoyalCH is the key factor in whether a customer ends up with CH or MM. At start, if LoyalCH is below 0.5036, customers lean toward MM, and if it's above 0.5036, they tend to choose CH. The tree then uses additional splits sometimes on LoyalCH again and sometimes on PriceDiff, SpecialCH, ListPriceDiff, and PctDiscMM to further divide the data. For example, customers with an extremely low LoyalCH (below 0.0356) almost always go for MM, while those with a very high LoyalCH (above 0.764572) almost exclusively choose CH. #Part e

```
ojPred <- predict(ojTree, newdata = ojTest, type = "class")
confMat <- table(ojTest$Purchase, ojPred)
print(confMat)
```

```
##      ojPred
##      CH  MM
## CH 160   8
## MM  38  64
```

```
errTest <- 1 - sum(diag(confMat)) / sum(confMat)
cat("Test error rate:", errTest, "\n")
```

```
## Test error rate: 0.1703704
```

The test error rate is about 17.04%. #Part f

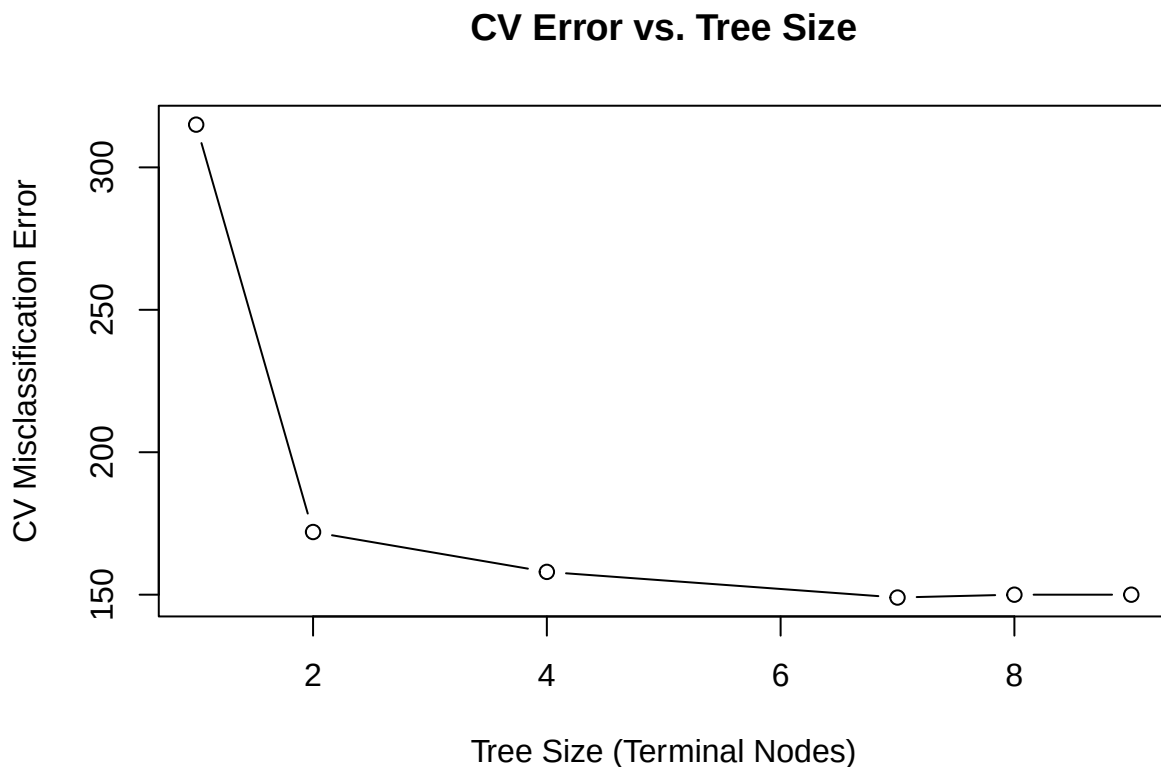
```
cvRes <- cv.tree(ojTree, FUN = prune.misclass)
print(cvRes)
```

```
## $size
## [1] 9 8 7 4 2 1
##
## $dev
## [1] 150 150 149 158 172 315
```

```
##
## $k
## [1]      -Inf    0.000000    3.000000    4.333333   10.500000  151.000000
##
## $method
## [1] "misclass"
##
## attr("class")
## [1] "prune"          "tree.sequence"
```

Tree with size 7 has the lowest deviance of 149. It is optimal. #Part g

```
plot(cvRes$size, cvRes$dev, type = "b",
     xlab = "Tree Size (Terminal Nodes)",
     ylab = "CV Misclassification Error",
     main = "CV Error vs. Tree Size")
```



At the leftmost point (with tree size = 1), the misclassification error is high. As the tree size increases to 2, 4, and then 7 nodes, the error drops significantly, reaching its lowest value around 7 terminal nodes (roughly 149). This is what we got in previous part. #Part h

```
optSize <- cvRes$size[which.min(cvRes$dev)]
cat("Optimal tree size:", optSize, "\n")
```

```
## Optimal tree size: 7
```

tree size of 7 corresponds to lowest cross-validated classification error rate. #Part i

```
prunedTree <- prune.misclass(ojTree, best = optSize)
prunedTree
```

```
## node), split, n, deviance, yval, (yprob)
##      * denotes terminal node
##
## 1) root 800 1073.00 CH ( 0.60625 0.39375 )
##    2) LoyalCH < 0.5036 365  441.60 MM ( 0.29315 0.70685 )
##      4) LoyalCH < 0.280875 177  140.50 MM ( 0.13559 0.86441 ) *
##      5) LoyalCH > 0.280875 188  258.00 MM ( 0.44149 0.55851 )
##        10) PriceDiff < 0.05 79   84.79 MM ( 0.22785 0.77215 ) *
##        11) PriceDiff > 0.05 109  147.00 CH ( 0.59633 0.40367 ) *
##    3) LoyalCH > 0.5036 435  337.90 CH ( 0.86897 0.13103 )
##      6) LoyalCH < 0.764572 174  201.00 CH ( 0.73563 0.26437 )
##        12) ListPriceDiff < 0.235 72   99.81 MM ( 0.50000 0.50000 )
##          24) PctDiscMM < 0.196196 55   73.14 CH ( 0.61818 0.38182 ) *
##          25) PctDiscMM > 0.196196 17   12.32 MM ( 0.11765 0.88235 ) *
##      13) ListPriceDiff > 0.235 102   65.43 CH ( 0.90196 0.09804 ) *
##    7) LoyalCH > 0.764572 261   91.20 CH ( 0.95785 0.04215 ) *
```

The above result is a pruned tree with five terminal nodes marked by *. #Part j

```
trainPredUnpruned <- predict(ojTree, newdata = ojTrain, type = "class")
errTrainUnpruned <- mean(trainPredUnpruned != ojTrain$Purchase)
trainPredPruned <- predict(prunedTree, newdata = ojTrain, type = "class")
errTrainPruned <- mean(trainPredPruned != ojTrain$Purchase)
cat("Training error (unpruned):", errTrainUnpruned, "\n")
```

```
## Training error (unpruned): 0.15875
```

```
cat("Training error (pruned):", errTrainPruned, "\n")
```

```
## Training error (pruned): 0.1625
```

The training error for the unpruned tree is about 15.9%, the pruned tree has a slightly higher error of about 16.3%. This small increase is expected as we simplified the model with fewer terminal nodes. #Part k

```
testPredUnpruned <- predict(ojTree, newdata = ojTest, type = "class")
errTestUnpruned <- mean(testPredUnpruned != ojTest$Purchase)
testPredPruned <- predict(prunedTree, newdata = ojTest, type = "class")
errTestPruned <- mean(testPredPruned != ojTest$Purchase)
cat("Test error (unpruned):", errTestUnpruned, "\n")
```

```
## Test error (unpruned): 0.1703704
```

```
cat("Test error (pruned):", errTestPruned, "\n")
```

```
## Test error (pruned): 0.162963
```

The test error for the unpruned tree is about 17.0%, while for the pruned tree has about 16.3% test error. As expected, pruning and simplifying the model decreased overfitting even though training error increased the test error decreased for the pruned tree compared to unpruned. Problem 4 #Part a

```
library(ISLR)
data(Hitters)
head(Hitters)
```

```
##           AtBat Hits HmRun Runs RBI Walks Years CAtBat CHits CHmRun
## -Andy Allanson    293   66    1   30  29   14    1    293   66    1
## -Alan Ashby       315   81    7   24  38   39   14   3449   835   69
## -Alvin Davis      479  130   18   66  72   76    3   1624   457   63
## -Andre Dawson     496  141   20   65  78   37   11   5628  1575  225
## -Andres Galarraga  321   87   10   39  42   30    2    396   101   12
## -Alfredo Griffin  594  169    4   74  51   35   11   4408  1133   19
##           CRuns CRBI CWalks League Division PutOuts Assists Errors
## -Andy Allanson     30   29    14      A        E    446    33    20
## -Alan Ashby       321  414   375      N        W    632    43    10
## -Alvin Davis      224  266   263      A        W    880    82    14
## -Andre Dawson     828  838   354      N        E    200    11     3
## -Andres Galarraga   48   46    33      N        E    805    40     4
## -Alfredo Griffin  501  336   194      A        W    282   421    25
##           Salary NewLeague
## -Andy Allanson      NA      A
## -Alan Ashby       475.0      N
## -Alvin Davis      480.0      A
## -Andre Dawson     500.0      N
## -Andres Galarraga   91.5      N
## -Alfredo Griffin  750.0      A
```

```
hitters <- na.omit(Hitters)
hitters$logSal <- log(hitters$Salary)
```

#Part b

```
train <- hitters[1:200, ]
test <- hitters[-(1:200), ]
```

#Part c

```
library(gbm)
```

```
## Loaded gbm 2.2.2
```

```
## This version of gbm is no longer under development. Consider transitioning to gbm3, https://github.com
```

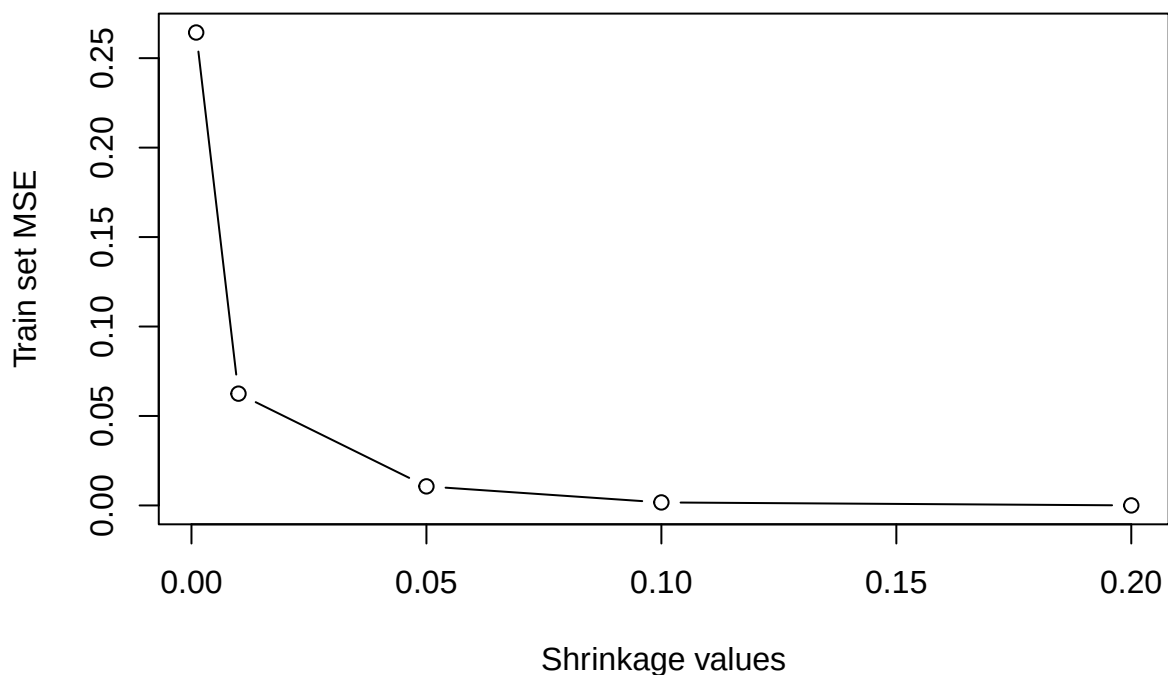
```
shrink <- c(0.001, 0.01, 0.05, 0.1, 0.2)
nTree <- 1000
trainMSE <- numeric(length(shrink))
for(i in seq_along(shrink)) {
  boostMod <- gbm(logSal ~ ., data = train[, !(names(train) %in% "Salary")],
```

```

        distribution = "gaussian", n.trees = nTree,
        shrinkage = shrink[i], interaction.depth = 4,
        verbose = FALSE)
predTrain <- predict(boostMod, newdata = train, n.trees = nTree)
trainMSE[i] <- mean((train$logSal - predTrain)^2)
}
plot(shrink, trainMSE, type = "b",
     xlab = "Shrinkage values",
     ylab = "Train set MSE",
     main = "Train set MSE vs. Shrinkage values")

```

Train set MSE vs. Shrinkage values



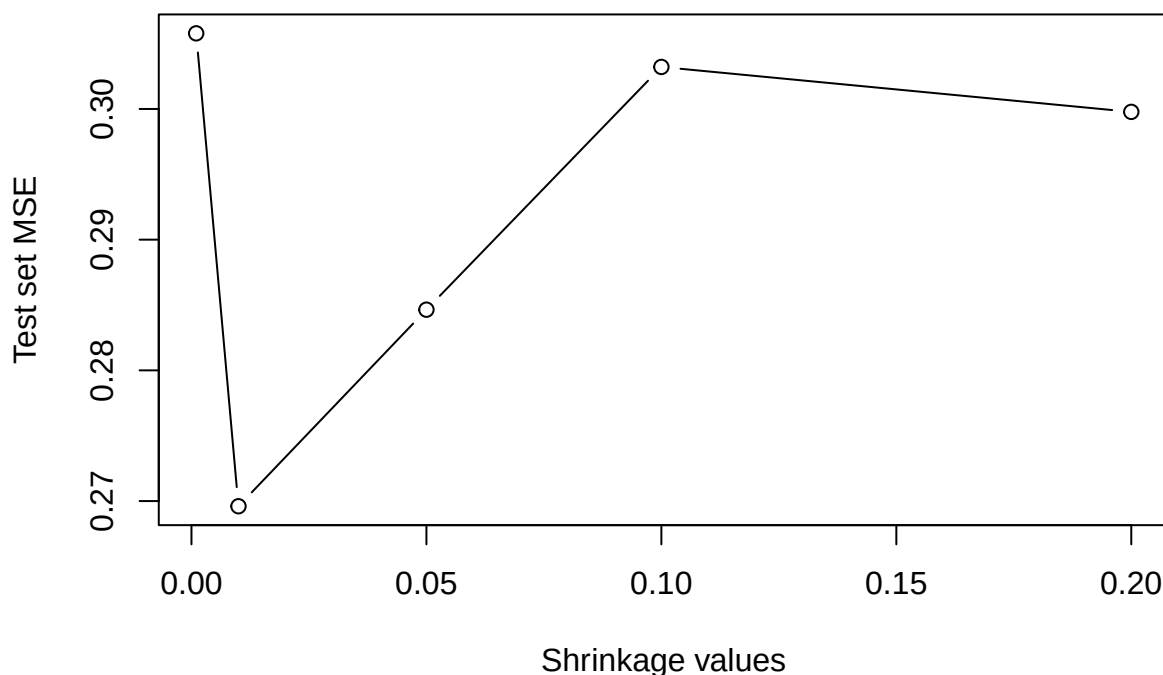
At very low shrinkage value, the model has higher training MSE. As shrinkage value grows, the MSE goes down significantly. #Part d

```

testMSE <- numeric(length(shrink))
for(i in seq_along(shrink)) {
  boostMod <- gbm(logSal ~ ., data = train[, !(names(train) %in% "Salary")],
    distribution = "gaussian", n.trees = nTree,
    shrinkage = shrink[i], interaction.depth = 4,
    verbose = FALSE)
  predTest <- predict(boostMod, newdata = test, n.trees = nTree)
  testMSE[i] <- mean((test$logSal - predTest)^2)
}
plot(shrink, testMSE, type = "b",
     xlab = "Shrinkage values",
     ylab = "Test set MSE",
     main = "Test set MSE vs. Shrinkage values")

```

Test set MSE vs. Shrinkage values



At very low and very high shrinkage value, the test MSE is high, while in the middle range the test MSE reaches its lowest point. This suggests that a moderate shrinkage leads to better performance of the model on test data. #Part e

```
library(glmnet)
```

```
## Loading required package: Matrix
```

```
## Loaded glmnet 4.1-8
```

```
xTrain <- model.matrix(logSal ~ ., data = train[, !(names(train) %in% "Salary")])[, -1]
yTrain <- train$logSal
xTest  <- model.matrix(logSal ~ ., data = test[, !(names(test) %in% "Salary")])[, -1]
yTest  <- test$logSal
grid <- 10^seq(10, -2, length = 100)

lassoMod <- glmnet(xTrain, yTrain, alpha = 1, lambda = grid) #Lasso
cvLasso <- cv.glmnet(xTrain, yTrain, alpha = 1)
bestLamLasso <- cvLasso$lambda.min
lassoPred <- predict(lassoMod, s = bestLamLasso, newx = xTest)
lassoMSE <- mean((yTest - lassoPred)^2)
cat("Lasso Test MSE:", lassoMSE, "\n")
```

```
## Lasso Test MSE: 0.470371
```

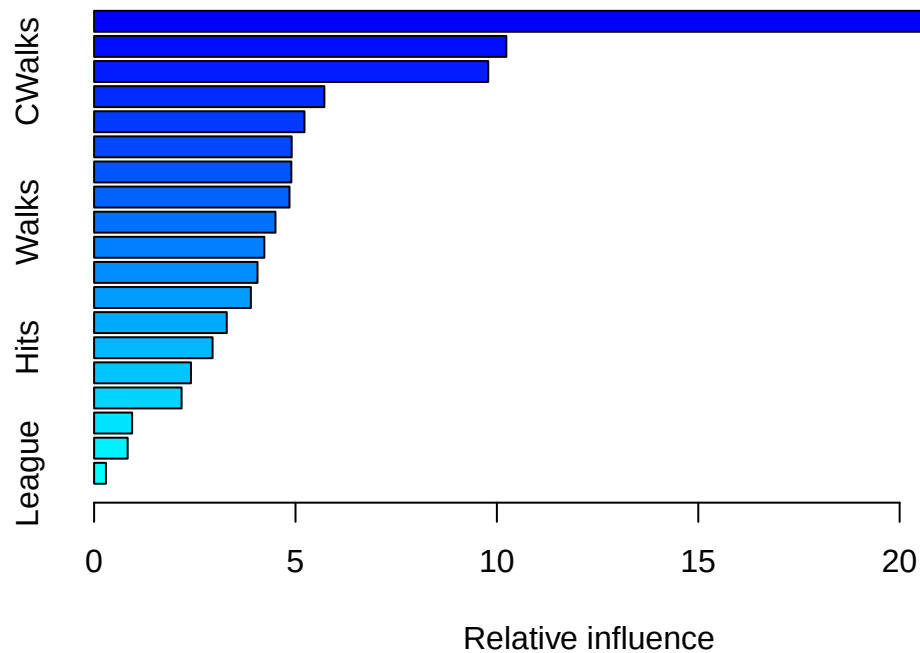
```
ridgeMod <- glmnet(xTrain, yTrain, alpha = 0, lambda = grid) #Ridge
cvRidge <- cv.glmnet(xTrain, yTrain, alpha = 0)
bestLamRidge <- cvRidge$lambda.min
```

```
ridgePred <- predict(ridgeMod, s = bestLamRidge, newx = xTest)
ridgeMSE <- mean((yTest - ridgePred)^2)
cat("Ridge Test MSE:", ridgeMSE, "\n")
```

```
## Ridge Test MSE: 0.4526352
```

The lasso model gives a test MSE of about 0.470, while the ridge model achieves a lower test MSE of roughly 0.450. This shows that, ridge regression is performing a bit better in predicting log-salary #Part f

```
summary(boostMod)
```



```
##          var    rel.inf
## CAtBat    CAtBat 24.8272012
## CRuns     CRuns 10.2350538
## CWalks    CWalks 9.7856503
## PutOuts   PutOuts 5.7166036
## Assists   Assists 5.2226422
## Years     Years  4.9058672
## AtBat     AtBat  4.8968059
## CHits     CHits  4.8503432
## Walks     Walks  4.5030290
## CHmRun    CHmRun 4.2265052
## RBI       RBI   4.0569258
## CRBI      CRBI  3.8916609
## HmRun     HmRun  3.2936541
## Hits      Hits  2.9412166
## Runs      Runs  2.4040786
## Errors    Errors 2.1715433
## Division  Division 0.9443481
## NewLeague NewLeague 0.8327215
## League    League  0.2941495
```

CAtBat is the most important predictor with a relative influence of approximately 29.5661340. CWalks, CHits, and CRBI also have high influence. Other variables like CRuns, Years, and PutOuts have lower relative influence. #Part g

```
library(randomForest)

## randomForest 4.7-1.2

## Type rfNews() to see new features/changes/bug fixes.

##
## Attaching package: 'randomForest'

## The following object is masked from 'package:ggplot2':
##
##      margin

numPred <- ncol(train) - 2
bagMod <- randomForest(logSal ~ ., data = train[, !(names(train) %in% "Salary")],
                      mtry = numPred, ntree = 500)
bagPred <- predict(bagMod, newdata = test)
bagMSE <- mean((test$logSal - bagPred)^2)
cat("Bagging Test set MSE:", bagMSE, "\n")

## Bagging Test set MSE: 0.2285834
```

The bagging model has a test MSE of about 0.225.